

Técnicas fundamentales de diseño de algoritmos

DIVIDIR Y CONQUISTAR

IIC2283 – Diseño y Análisis de Algoritmos

Departamento de Ciencia de la Computación
Pontificia Universidad Católica de Chile

2025-2

Técnicas de diseño de algoritmos

En este capítulo vamos a estudiar tres técnicas fundamentales para el diseño de algoritmos:

- ▶ Dividir y conquistar
- ▶ Programación dinámica
- ▶ Algoritmos greedy (codiciosos)

Estas tres técnicas son útiles para desarrollar algoritmos en muchos escenarios.

Dividir y conquistar

- ▶ La técnica de diseño de algoritmos *dividir y conquistar* permite resolver un problema mediante su división en **subproblemas** que **no se solapan** entre sí.
- ▶ Cada subproblema es una instancia más pequeña del problema original, y es resuelta recursivamente usando el mismo algoritmo.
- ▶ Al ser más pequeños, los subproblemas son más “fáciles” de resolver.
- ▶ El proceso se repite recursivamente, dividiendo en subproblemas cada vez más pequeños, hasta que sean lo suficientemente pequeños para resolverlos de manera simple.

Dividir y conquistar

Un algoritmo del tipo dividir y conquistar consiste en las siguientes etapas:

- ▶ **Caso Base:** Si la instancia a resolver es suficientemente pequeña, se resuelve directamente.
- ▶ **Dividir:** el problema original de tamaño $|w| = n$ es dividido en ℓ subproblemas $w_1, \dots, w_\ell$ de tamaños $n_1, \dots, n_\ell$, respectivamente.
- ▶ **Conquistar:** los subproblemas $w_1, \dots, w_\ell$ se resuelven recursivamente, obteniendo soluciones $S_1, \dots, S_\ell$.
- ▶ **Combinar:** las soluciones $S_1, \dots, S_\ell$ se combinan para formar la solución al problema original w .

Dividir y conquistar

Esta es la forma genérica de un algoritmo que utiliza la técnica de dividir para conquistar:

```
DividirParaConquistar( $w$ )  
  if  $|w| \leq k$  then return InstanciasPequeñas( $w$ )  
  else  
    Dividir  $w$  en  $w_1, \dots, w_\ell$   
    for  $i := 1$  to  $\ell$  do  
       $S_i :=$  DividirParaConquistar( $w_i$ )  
    return Combinar( $S_1, \dots, S_\ell$ )
```

Dividir y conquistar

- ▶ En **DividirParaConquistar** k es una constante que indica cuándo el tamaño de una entrada w es considerado pequeño: $|w| \leq k$
- ▶ Las entradas pequeñas son solucionadas utilizando un algoritmo diseñado para ellas: **InstanciasPequeñas**
 - ▶ En general este algoritmo es sencillo y ejecuta un número pequeño de operaciones
- ▶ Si el tamaño de una entrada w no es pequeño ($|w| > k$), entonces w es dividido en una secuencia $w_1, \dots, w_\ell$ de entradas de menor tamaño para **DividirParaConquistar**

Dividir y conquistar

- ▶ Para cada $i \in \{1, \dots, \ell\}$ la llamada **DividirParaConquistar**(w_i) resuelve el problema para la entrada w_i
 - ▶ El resultado de esta llamada es almacenado en S_i
- ▶ Finalmente la llamada **Combinar**($S_1, \dots, S_\ell$) combina los resultados $S_1, \dots, S_\ell$ para obtener el resultados para la instancia original w

Ejemplos típicos que usan esta técnica:

- ▶ MergeSort
- ▶ QuickSort
- ▶ Búsqueda binaria
- ▶ Estudiamos a continuación esta técnica para obtener un algoritmo eficiente para la multiplicación de dos números enteros

Dividir y conquistar: el problema de multiplicar dos enteros

Supongamos que queremos multiplicar dos números enteros positivos x e y , de n dígitos cada uno.

- ▶ Esta es una operación aritmética fundamental enseñada en la escuela básica.
- ▶ Esta operación es incluida como instrucción en la mayoría de procesadores.
 - ▶ Es decir, los procesadores pueden calcular en tiempo constante $x \times y$.
 - ▶ Pero para números con una cantidad limitada de dígitos, típicamente $n = 32$ o $n = 64$ bits.
- ▶ El problema se vuelve no trivial cuando n es grande, algo típico en:
 - ▶ Criptografía.
 - ▶ Cálculo de constantes clásicas como π ó e .

Antes de multiplicar: suma de números enteros

Sean a y b dos números enteros con $n \geq 1$ dígitos cada uno.

Primero queremos obtener $c = a + b$

Vamos a utilizar el algoritmo usual para calcular c

$$\begin{array}{r} 9 7 4 3 1 \\ + 1 4 0 3 6 \\ \hline 1 1 1 4 6 7 \end{array}$$

- ▶ Consideramos la **suma de dos dígitos**, la **comparación de dos dígitos** y la **resta de un número con a lo más dos dígitos con uno de un dígito** como las operaciones a contar, cada una con costo 1.
- ▶ ¿Cuántos dígitos puede tener c ?
- ▶ El algoritmo de suma realiza $\Theta(n)$ operaciones

Multiplicación de números enteros

Ahora queremos obtener $d = x \times y$

Primero recordemos el algoritmo clásico para calcular $x \times y$

$$\begin{array}{r} 97431 \\ \times 14036 \\ \hline 584586 \\ 292293 \\ 000000 \\ 389724 \\ + 97431 \\ \hline 1367541516 \end{array}$$

- ▶ Consideramos la suma y la multiplicación de dígitos como las operaciones a contar, ambas con costo 1
- ▶ El algoritmo de multiplicación clásico realiza $\Theta(n^2)$ operaciones
- ▶ Imagine tener que multiplicar dos enteros de, digamos, 1 millón de dígitos cada uno $\rightarrow 10^{12}$ operaciones

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

Hasta 1960, se pensaba que no era posible multiplicar dos números de n dígitos ejecutando menos que n^2 operaciones

Incluso Kolmogorov lo pensaba e intentó demostrarlo

En 1960 organizó un seminario en el que estaba presente Karatsuba (un estudiante en ese momento), quien resolvió la pregunta



Kolmogorov



Karatsuba

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

- ▶ Denotemos con $x[n..1]$ e $y[n..1]$ a los números que queremos multiplicar, asumiendo que n es una potencia de 2.
 - ▶ $x[n]$ e $y[n]$ son los dígitos **más** significativos.
 - ▶ $x[1]$ e $y[1]$ son los dígitos **menos** significativos
- ▶ Vamos a denotar con $x_S = x[n..\frac{n}{2} + 1]$ y $x_I[\frac{n}{2}..1]$, a la mitad superior (S) e inferior (I) de x , respectivamente. Hacemos lo mismo para y .

$$x[n..1] = \underbrace{x[n]x[n-1]\cdots x\left[\frac{n}{2}+1\right]}_{x_S} \underbrace{x\left[\frac{n}{2}\right]x\left[\frac{n}{2}-1\right]\cdots x[1]}_{x_I}$$

$$y[n..1] = \underbrace{y[n]y[n-1]\cdots y\left[\frac{n}{2}+1\right]}_{y_S} \underbrace{y\left[\frac{n}{2}\right]y\left[\frac{n}{2}-1\right]\cdots y[1]}_{y_I}$$

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

- ▶ Por ejemplo, si $x = 56824385$ e $y = 96588545$, entonces:
 - ▶ $x_S = 5682$ y $x_I = 4385$.
 - ▶ $y_S = 9658$ y $y_I = 8545$.
- ▶ Note que:
 - ▶ $x = 10^{n/2}x_S + x_I$.
 - ▶ $y = 10^{n/2}y_S + y_I$.
- ▶ Para el ejemplo anterior:
 - ▶ $x = 10^4x_S + x_I = 56820000 + 4385 = 56824385$,
 - ▶ $y = 10^4y_S + y_I = 96580000 + 8545 = 96588545$.

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

- ▶ Usando esto, escribimos la multiplicación de la siguiente manera:

$$\begin{aligned}x \times y &= (10^{n/2}x_S + x_I) \times (10^{n/2}y_S + y_I) \\&= 10^n x_S \times y_S + 10^{n/2}(x_S \times y_I + y_S \times x_I) + x_I \times y_I\end{aligned}$$

- ▶ Eso implica las siguientes multiplicaciones recursivas:
 - ▶ $x_S \times y_S$
 - ▶ $x_S \times y_I$
 - ▶ $y_S \times x_I$
 - ▶ $x_I \times y_I$
- ▶ El resto de las operaciones toman tiempo $\Theta(n)$:
 - ▶ Sumas
 - ▶ Multiplicaciones por una potencia de 10 (solo se agregan ceros a la derecha)

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

- Analizamos a continuación nuestro algoritmo dividir y conquistar:

$$x \times y = 10^n x_S \times y_S + 10^{n/2} (x_S \times y_I + y_S \times x_I) + x_I \times y_I$$

Algoritmo 1: $\text{MULT}(x[n..1], y[n..1])$

```
1 if  $n = 1$  then
2   | return  $x[1] \cdot y[1]$ 
3 else
4   | // Dividir
5   |  $x_I \leftarrow x[\frac{n}{2}..1], \quad x_S \leftarrow x[n..\frac{n}{2} + 1]$ 
6   |  $y_I \leftarrow y[\frac{n}{2}..1], \quad y_S \leftarrow y[n..\frac{n}{2} + 1]$ 
7   | // Conquistar
8   |  $p_1 \leftarrow \text{MULT}(x_S, y_S), \quad p_2 \leftarrow \text{MULT}(x_S, y_I)$ 
9   |  $p_3 \leftarrow \text{MULT}(y_S, x_I), \quad p_4 \leftarrow \text{MULT}(x_I, y_I)$ 
10  | // Combinar
11  | return  $10^n \cdot p_1 + 10^{n/2} \cdot (p_2 + p_3) + p_4$ 
12 end
```

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

El tiempo de ejecución es:

$$T(n) = \begin{cases} 1, & n = 1 \\ 4T\left(\frac{n}{2}\right) + \Theta(n), & n > 1 \end{cases}$$

De acuerdo al Teorema Maestro, tenemos que $T(n) \in \Theta(n^2)$

Esto no mejora en nada al algoritmo clásico, pero es un buen punto de comienzo

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

Karatsuba notó lo siguiente (inspirado en un truco de Gauss)

$$x \times y = 10^n x_S \times y_S + 10^{n/2} \left(\underbrace{x_S \times y_I + y_S \times x_I}_{(x_S + x_I) \times (y_S + y_I) - x_S \times y_S - x_I \times y_I} \right) + x_I \times y_I$$

Ahora podemos calcular:

- ▶ $p_1 \leftarrow x_S \times y_S$
- ▶ $p_2 \leftarrow x_I \times y_I$
- ▶ $p_3 \leftarrow (x_S + x_I) \times (y_S + y_I)$

Luego tenemos:

$$x \times y = 10^n \cdot p_1 + 10^{n/2} \cdot (p_3 - p_1 - p_2) + p_2$$

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

Algoritmo 2: KARATSUBA($x[n..1]$, $y[n..1]$)

```
1 if  $n = 1$  then
2   |   return  $x[1] \cdot y[1]$ 
3 else
4   |   // Dividir
5   |    $x_I \leftarrow x[\frac{n}{2}..1], \quad x_S \leftarrow x[n..\frac{n}{2} + 1]$ 
6   |    $y_I \leftarrow y[\frac{n}{2}..1], \quad y_S \leftarrow y[n..\frac{n}{2} + 1]$ 
7   |   // Conquistar
8   |    $p_1 \leftarrow \text{KARATSUBA}(x_S, y_S)$ 
9   |    $p_2 \leftarrow \text{KARATSUBA}(x_I, y_I)$ 
10  |    $p_3 \leftarrow \text{KARATSUBA}(x_S + x_I, y_S + y_I)$ 
11  |   // Combinar
12  |   return  $10^n \cdot p_1 + 10^{n/2} \cdot (p_3 - p_1 - p_2) + p_2$ 
13 end
```

Dividir y conquistar: el algoritmo de multiplicación de Karatsuba

La cantidad de operaciones llevadas a cabo por el algoritmo de Karatsuba corresponde a la siguiente ecuación de recurrencia:

$$T(n) = \begin{cases} 1, & n = 1 \\ 3T\left(\frac{n}{2}\right) + \Theta(n), & n > 1. \end{cases}$$

Usando el Teorema Maestro, tenemos que $T(n) \in \Theta(n^{\log_2 3})$

Dado que $\log_2 3 \approx 1,585$, esto representa una notable mejora respecto al algoritmo clásico

- ▶ Mejora por un factor de $\Theta(n^{0,415})$
- ▶ Para $n = 1,000,000$, tiempo proporcional a $\sim 3,2 \times 10^9$

Karatsuba para n general

¿Cómo debe formularse el algoritmo de Karatsuba en el caso general, cuando n no es potencia de 2? ¿Cuál es la complejidad de este algoritmo?

- ▶ La clave para realizar el análisis del algoritmo de Karatsuba es el uso de ecuaciones de recurrencia e inducción constructiva

En el caso general, representamos las entradas x e y de la siguiente forma:

$$\begin{aligned}x &= 10^{\lfloor \frac{n}{2} \rfloor} \cdot x_S + x_I \\y &= 10^{\lfloor \frac{n}{2} \rfloor} \cdot y_S + y_I\end{aligned}$$

Karatsuba para n general

La siguiente ecuación de recurrencia para $T(n)$ modela la cantidad de operaciones realizadas por el algoritmo (para constantes a, b):

$$T(n) = \begin{cases} a & n = 1 \\ T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + T(\lceil \frac{n}{2} \rceil + 1) + b \cdot n & n > 1. \end{cases}$$

Ejercicio

Demuestre usando inducción constructiva que $T(n)$ es $O(n^{\log_2(3)})$

► En particular, demuestre lo siguiente:

$$(\exists c \in \mathbb{R}^+)(\exists d \in \mathbb{R}^+)(\exists n_0 \in \mathbb{N})(\forall n \geq n_0)(T(n) \leq c \cdot n^{\log_2(3)} - d \cdot n)$$

El Problema de la Mayoría Absoluta

Pensemos en un sistema que permite realizar votaciones:

- ▶ n usuarios pueden votar por una de entre k opciones posibles.
- ▶ Al finalizar la votación, el sistema debe informar cuál fue la opción más votada e indicar si hubo mayoría absoluta o no.

Hay mayoría absoluta si la opción ganadora recibió al menos $\lfloor \frac{n}{2} \rfloor + 1$ votos.

- ▶ El problema puede plantearse abstractamente así:

Dado un arreglo $A[1..n]$ de elementos de algún tipo (no necesariamente ordenado), determinar si existe un elemento que se repite al menos $\lfloor \frac{n}{2} \rfloor + 1$ veces en A .

El Problema de la Mayoría Absoluta

- ▶ Si las alternativas a votar están representadas por elementos de tipo entero en el rango $[1..k]$.
 - ▶ Se usa un arreglo de contadores $C[1..k]$.
 - ▶ Se recorre el arreglo A , incrementando $C[A[i]]$ por cada $i = 1, \dots, n$.
 - ▶ Finalmente, se recorre C para determinar si alguna de las opciones es mayoría absoluta.
- ▶ El tiempo total es $\Theta(n + k)$, mientras el espacio adicional utilizado $\Theta(k)$.
- ▶ No funciona si las alternativas a votar no son enteros en $[1..k]$

El Problema de la Mayoría Absoluta

Para objetos sobre los que existe una relación de orden total pero no necesariamente están en el rango $[1..k]$ (por ejemplo, strings).

- ▶ Una alternativa simple sería ordenar el arreglo A .
- ▶ Note que si A tiene una mayoría, este sería el elemento $A[n/2]$ después de ordenar.
 - ▶ Si un elemento es mayoría, sus $\lfloor \frac{n}{2} \rfloor + 1$ ocurrencias siempre van a ocupar más de la mitad del arreglo ordenado.
 - ▶ Para determinar si $A[n/2]$ es mayoría solo debemos recorrer el arreglo contando las ocurrencias de éste.
- ▶ Si usamos MergeSort para ordenar A , el tiempo de ejecución será $\Theta(n \log n)$ y el espacio adicional usado es $\Theta(n)$.

El Problema de la Mayoría Absoluta

Todas las propuestas tienen las siguientes desventajas:

- ▶ No son in-place: se requiere espacio adicional no constante, ya sea para el arreglo C o para ordenar usando MergeSort.
- ▶ Asume el modelo de comparaciones, y la existencia de una relación de orden total sobre los elementos del arreglo.
 - ▶ En algunas aplicaciones podríamos tener un conjunto parcialmente ordenado (los elementos se pueden comparar mediante $=$ o $\neq$).
 - ▶ Por ejemplo, que cada elemento del conjunto A sea otro conjunto.
- ▶ Estudiamos a continuación una solución Dividir y Conquistar, que usa espacio adicional $\Theta(\log n)$ y que solo necesita comparar elementos usando $=$.

El Problema de la Mayoría Absoluta: Dividir y Conquistar

Construimos a continuación el algoritmo recursivo de tipo Dividir y Conquistar:

- ▶ **Caso Base:** Note que para $n = 1$, note que el único elemento del arreglo es una mayoría, ya que aparece $\lfloor \frac{1}{2} \rfloor + 1 = 1$ veces
- ▶ **Hipótesis Inductiva:** asumimos que $\text{MAYORÍA}(A[1..n'])$ computa correctamente el elemento mayoritario del arreglo A , en caso de existir, $\forall n' \in \{1, \dots, n-1\}$. En el caso en que el arreglo no tenga elemento mayoritario, el algoritmo indica esta situación
- ▶ **Paso Inductivo:** para $n \geq 2$, mostramos que podemos construir la solución para $\text{MAYORÍA}(A[1..n])$ en base a saber resolver casos más pequeños

El Problema de la Mayoría Absoluta: Dividir y Conquistar

Paso Inductivo (cont.)

- ▶ ¿Qué pasa si dividimos el arreglo A en 2 mitades de $n/2$ elementos cada uno? (podemos asumir n potencia de 2 para simplificar)
- ▶ Por hipótesis inductiva, si hacemos $\text{MAYORÍA}(A[1..n/2])$ y $\text{MAYORÍA}(A[n/2 + 1..n])$ habremos obtenido los elementos mayoritarios de ambas mitades del arreglo
- ▶ ¿De qué nos sirve resolver el problema en las mitades del arreglo?
- ▶ El siguiente Lema lo explica

El Problema de la Mayoría Absoluta: Dividir y Conquistar

Lema

Si a es una mayoría en el arreglo $A[1..n]$, entonces a es también una mayoría en al menos una de las dos mitades del arreglo, $A[1..\frac{n}{2}]$ ó $A[\frac{n}{2} + 1..n]$.

Esto significa que si dividimos el arreglo en dos mitades podría darse el caso que:

- ▶ a ocurre $\lfloor \frac{n}{4} \rfloor$ veces en una de las mitades, y
- ▶ $\lfloor \frac{n}{4} \rfloor + 1$ veces en la otra (lo que la haría mayoría en esa mitad).
- ▶ Note que el recíproco no es necesariamente cierto.
- ▶ Nuestro algoritmo Dividir y Conquistar funciona de la siguiente manera.

El Problema de la Mayoría Absoluta: Dividir y Conquistar

Paso Inductivo (cont.)

- ▶ Usamos el resultado del Lema para construir la solución de la siguiente manera
- ▶ Sea $m_1 \leftarrow \text{MAYORÍA}(A[1..n/2])$
- ▶ Sea $m_2 \leftarrow \text{MAYORÍA}(A[n/2 + 1..n])$
- ▶ Esos dos valores son candidatos a ser mayoría (según el Lema, si A tiene mayoría, ésta tiene que ser m_1 o m_2)
- ▶ La etapa de combinar consiste en verificar si m_1 o m_2 son mayoría, simplemente recorriendo el arreglo y contando la cantidad de ocurrencias
- ▶ Si uno de esos valores ocurre al menos $\lfloor \frac{n}{2} \rfloor + 1$ veces, entonces se reporta como mayoría.

El Problema de la Mayoría Absoluta: Dividir y Conquistar

Algoritmo 31: MAYORÍA($A[1..n]$)

```
1 if  $n = 1$  then
2   | return  $A[1]$ 
3 else
4   |  $m_1 \leftarrow \text{MAYORÍA}(A[1..\frac{n}{2}])$ 
5   |  $m_2 \leftarrow \text{MAYORÍA}(A[\frac{n}{2} + 1..n])$ 
6   |  $c_1 \leftarrow \text{máx}\{\text{CONTAR}(A[1..n], m_1), 0\}$ 
7   |  $c_2 \leftarrow \text{máx}\{\text{CONTAR}(A[1..n], m_2), 0\}$ 
8   | if  $c_1 \geq \lfloor \frac{n}{2} \rfloor + 1$  then
9   |   | return  $m_1$ 
10  | else
11  |   | if  $c_2 \geq \lfloor \frac{n}{2} \rfloor + 1$  then
12  |   |   | return  $m_2$ 
13  |   | else
14  |   |   | return NoHayMayoría
15  |   | end
16  | end
17 end
```

Algoritmo 32: CONTAR($A[1..n], m$)

```
1 if  $m = \text{NoHayMayoría}$  then
2   | return  $-\infty$ 
3 else
4   |  $c \leftarrow 0$ 
5   | for  $i \leftarrow 1$  to  $n$  do
6   |   | if  $A[i] = m$  then
7   |   |   |  $c++$ 
8   |   | end
9   | end
10  | return  $c$ 
11 end
```

El Problema de la Mayoría Absoluta: Tiempo de Ejecución

- ▶ Note que CONTAR toma tiempo $O(n)$ (medido en cantidad de comparaciones de igualdad).
- ▶ Por lo tanto, el tiempo de ejecución es:

$$T(n) = \begin{cases} 2T(\frac{n}{2}) + O(n), & n \geq 2; \\ 1, & n = 1. \end{cases}$$

- ▶ Por Teorema del Maestro significa que $T(n) \in O(n \log n)$.
- ▶ El mismo tiempo que ordenar el arreglo, con espacio adicional $O(\log n)$ (altura del árbol de recursión).

Una aplicación en geometría computacional

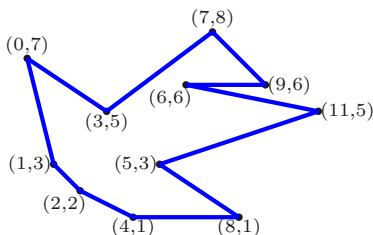
- ▶ La geometría computacional estudia problemas geométricos desde un punto de vista algorítmico
- ▶ Estudiaremos un caso de aplicación de la técnica Dividir y Conquistar en geometría computacional
- ▶ Como tal, un algoritmo geométrico necesita representar de alguna manera los objetos básicos de la geometría, por ejemplo:
 - ▶ Un punto en el plano puede representarse por un par de números indicando sus coordenadas: (x, y)
 - ▶ Un segmento de línea $\overline{p_i p_j}$ entre los puntos p_i y p_j en el plano es representado por 2 puntos: $p_i = (x_i, y_i)$ y $p_j = (x_j, y_j)$.
 - ▶ Un polígono en el plano es un conjunto ordenado de puntos, como vemos en detalle a continuación

Una aplicación en geometría computacional

Un polígono $\mathcal{P}$ de n puntos en el plano es una secuencia de puntos $\langle p_1, p_2, \dots, p_n \rangle$, tal que:

- ▶ para $i = 1, \dots, n - 1$ se cumple que $\overline{p_i p_{i+1}}$ es un lado del polígono, y
- ▶ $\overline{p_n p_1}$ es un lado del polígono.

Ejemplo, un polígono representado por la secuencia de puntos $\langle (2, 2), (4, 1), (8, 1), (5, 3), (11, 5), (6, 6), (9, 6), (7, 8), (3, 5), (0, 7), (1, 3) \rangle$.



Una aplicación en geometría computacional

- ▶ Es natural representar dicha secuencia de puntos usando una lista **circular** $L = \langle p_1, \dots, p_n \rangle$.
 - ▶ Circular: que el último elemento tiene como sucesor al primer elemento de la lista
- ▶ Cualquiera de los puntos del polígono puede ser elegido como el primer elemento de la lista —por la circularidad de la misma.
 - ▶ Lo importante es que los puntos están en secuencia, en donde cada par de puntos consecutivos definen un lado del polígono
- ▶ Puede usarse cualquiera de los dos sentidos posibles para los puntos: horario o antihorario

Una aplicación en geometría computacional

- ▶ También es necesario contar con tests que permitan chequear ciertas propiedades geométricas de interés, como por ejemplo el siguiente
- ▶ Sean $p_1 = (x_1, y_1)$, $p_2 = (x_2, y_2)$, y $p_3 = (x_3, y_3)$.
- ▶ El área (con signo) del triángulo formado por esos tres puntos puede calcularse con el determinante:

$$\begin{aligned} \text{AS}(\Delta(p_1, p_2, p_3)) &= \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix} \\ &= x_1y_2 + x_2y_3 + x_3y_1 - x_1y_3 - x_2y_1 - x_3y_2. \end{aligned}$$

- ▶ Note que calcular el determinante toma tiempo $O(1)$

Una aplicación en geometría computacional

Lo que nos interesa de la anterior expresión es su signo, ya que::

- ▶ $AS(\Delta(p_1, p_2, p_3)) > 0$: p_3 a la izquierda de $\overrightarrow{p_1 p_2}$, $\Rightarrow$ ángulo interno en p_2 es convexo.
- ▶ $AS(\Delta(p_1, p_2, p_3)) < 0$: p_3 a la derecha de $\overrightarrow{p_1 p_2}$, $\Rightarrow$ ángulo interno en p_2 es cóncavo.
- ▶ $AS(\Delta(p_1, p_2, p_3)) = 0$: p_1 , p_2 , y p_3 son puntos colineales, esto es, el ángulo interno en p_2 es llano.

Ejercicio

Escriba un algoritmo que determine si un polígono dado es convexo o no, en tiempo $\Theta(n)$.

Una aplicación en geometría computacional

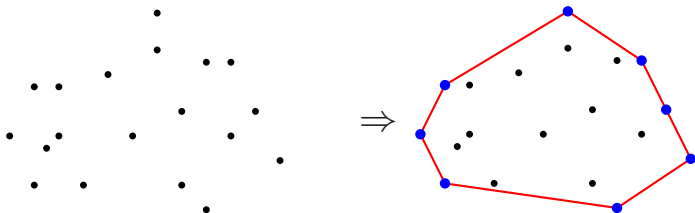
- ▶ A continuación resolvemos un problema fundamental en geometría computacional, usando Dividir y Conquistar
- ▶ El convex hull de un conjunto de puntos

Dado un conjunto $P = \{p_1, \dots, p_n\}$ de n puntos en el plano, el *convex hull* (*cierre convexo*, o *cápsula convexa*) $\mathcal{CH}(P)$ es el conjunto convexo de área mínima que contiene a P .

- ▶ Un conjunto de infinitos puntos S en el plano es convexo si para cualquier par de puntos $p, q \in S$, el segmento $\overline{pq}$ está completamente contenido dentro de S .

Una aplicación en geometría computacional

- ▶ Imagine que los puntos en el plano son clavos sobre una madera, de manera que sus cabezas sobresalen
- ▶ Imagine también una banda elástica que se expande con una mano para que contenga a todos los clavos, y luego se suelta para que el elástico se mueva hacia los clavos.
- ▶ El elástico se va a apoyar sobre los clavos “extremos” del conjunto, definiendo un polígono que se apoya sobre esos puntos extremos.

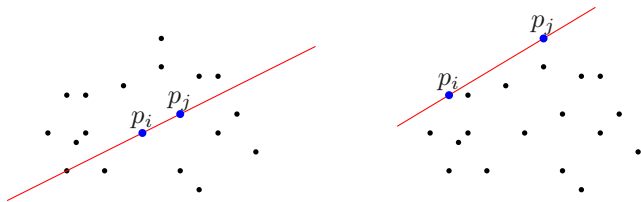


Una aplicación en geometría computacional

La siguiente propiedad es clave para calcular $\mathcal{CH}(P)$:

Lemma

Dado un conjunto de puntos $P = \{p_1, \dots, p_n\}$ en el plano, el segmento $\overline{p_i p_j}$ (para cualquier par de puntos $p_i \neq p_j$) pertenece a $\mathcal{CH}(P)$ si y sólo si al considerar los dos semiplanos determinados por la recta infinita que pasa por p_i y p_j , todos los puntos de P se ubican en un mismo semiplano.



Resultado: Algoritmo fuerza bruta de tiempo $O(n^3)$

Una aplicación en geometría computacional

Esta propiedad nos permite definir el siguiente enfoque de fuerza bruta:

- ▶ por cada posible par de puntos p_i y p_j del conjunto, chequear que el resto de los puntos de P se ubiquen en el mismo semiplano respecto a la recta infinita que pasa por p_i y p_j .
- ▶ Si eso se cumple, agregar el segmento $\overline{p_i p_j}$ al convex hull
- ▶ Aquí es necesario usar el test del determinante visto en la clase anterior

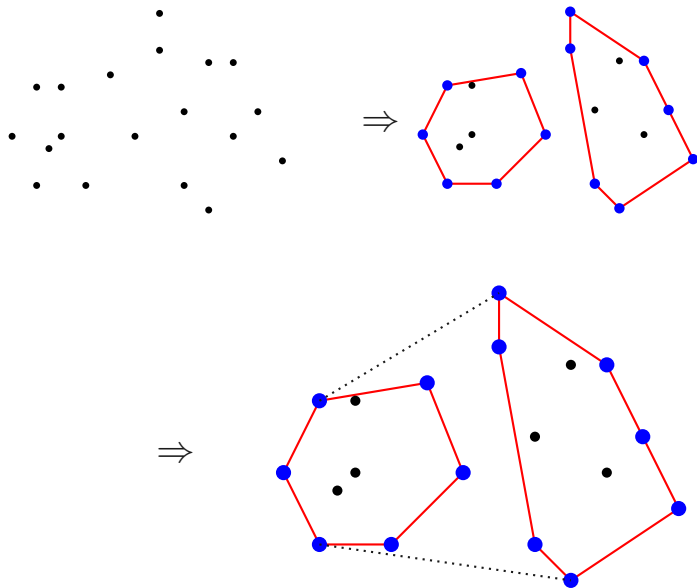
Resultado: Algoritmo fuerza bruta de tiempo $O(n^3)$:

- ▶ $\sim n^2/2$ pares de puntos distintos
- ▶ Por cada par de puntos, recorrer el resto de puntos chequeando que si todos quedan del mismo lado o no

Una aplicación en geometría computacional

- ▶ Estudiamos a continuación un enfoque Dividir y Conquistar para resolver el problema
- ▶ Rationale: obtener el convex hull de conjuntos de puntos pequeños es más fácil que para conjuntos más grandes
 - ▶ Por ejemplo, para un conjunto de 3 puntos es trivial calcular su convex hull: el triángulo definido por los tres puntos
- ▶ Sólo necesitaremos poder combinar convex hulls

Una aplicación en geometría computacional



Una aplicación en geometría computacional

Asumimos que los puntos han sido ordenados por coordenada x creciente antes de invocar el algoritmo por primera vez

Algoritmo 3: $\text{MERGEHULL}(P = \{p_1, p_2, \dots, p_n\})$

```
1 if  $|P| \leq 5$  then
2   return CONVEXHULLFUERZABRUTA( $P$ )
3 else
4   // Dividir
5    $P_1 \leftarrow \{p_1, \dots, p_{\lfloor n/2 \rfloor}\}$ 
6    $P_2 \leftarrow \{p_{\lfloor n/2 \rfloor + 1}, \dots, p_n\}$ 
7   // Conquistar
8    $C_1 \leftarrow \text{MERGEHULL}(P_1)$ 
9    $C_2 \leftarrow \text{MERGEHULL}(P_2)$ 
10  // Combinar
11  return COMBINARHULLS( $C_1, C_2$ )
12 end
```

Una aplicación en geometría computacional

- ▶ La etapa más compleja es la de combinar los convex hulls ya calculados para las mitades del conjunto de puntos
- ▶ La idea es calcular dos tangentes (una superior y otra inferior) para los dos conjuntos de puntos
- ▶ Note que, por la forma en que funciona el algoritmo, los dos convex hulls C_1 y C_2 son disjuntos y separables por una línea vertical
- ▶ Eso simplifica un poco el cálculo de las tangentes
- ▶ El siguiente algoritmo de Preparata y Hong (1977) permite encontrar esas tangentes en tiempo $\Theta(n)$

Una aplicación en geometría computacional

- ▶ El algoritmo asume que C_1 y C_2 se representan usando una lista circular ordenada en sentido antihorario
- ▶ Para un vértice p del convex hull, $p + 1$ es el siguiente vértice en sentido antihorario, mientras que $p - 1$ es el anterior vértice

Algoritmo 4: COMBINARHULLS(C_1, C_2)

```
1  $p_1 \leftarrow$  punto de mas a la derecha en  $C_1$ 
2  $p_2 \leftarrow$  punto de mas a la izquierda en  $C_2$ 
3 // Tangente inferior
4 while  $\overrightarrow{p_1 p_2}$  no es una tangente inferior de  $C_1$  y  $C_2$  do
5   while  $\overrightarrow{p_1 p_2}$  no es una tangente de  $C_1$  do
6      $p_1 \leftarrow p_1 - 1$ 
7   end
8   while  $\overrightarrow{p_1 p_2}$  no es una tangente de  $C_2$  do
9      $p_2 \leftarrow p_2 + 1$ 
10  end
11 end
12 // Tangente superior, se calcula de forma similar
```

Una aplicación en geometría computacional

- ▶ El tiempo de ejecución de MERGEHULL es:

$$T(n) = \begin{cases} a, & n \leq 5 \\ 2T(\frac{n}{2}) + \Theta(n), & n > 5 \end{cases}$$

- ▶ Por el Teorema Maestro, el tiempo de ejecución es $T(n) \in \Theta(n \log n)$
- ▶ Se puede demostrar que cualquier algoritmo para calcular el convex hull de un conjunto de n puntos puede ser usado para ordenar un conjunto de n elementos
- ▶ **Conclusión:** Ningún algoritmo para calcular el convex hull puede hacerlo en tiempo menor a $\Theta(n \log n)$ en el peor caso, por lo que MERGEHULL es óptimo